

# INFORME DE AUDITORÍA TÉCNICA INTEGRAL

Evaluación de Flujo Funcional (Frontend & Backend) y Brechas de Seguridad (OWASP Top 10)

|                                                                 |                                                                          |
|-----------------------------------------------------------------|--------------------------------------------------------------------------|
| Sistema Evaluado: WebPrediccionDT2 (Laravel 11 + Supabase)      | Fecha de Auditoría: 27 de Septiembre, 2026                               |
| Microservicio ML: AppML-Tesis (Python Flask + Random Forest)    | Entorno de Ejecución: Vercel Serverless / AWS Pooler                     |
| Normativas de Referencia: OWASP Top 10:2021   HIPAA / Ley 29733 | Dictamen Global: <b>RIESGO ALTO (Requiere Parches RBAC &amp; Config)</b> |

## 1. ARQUITECTURA Y FLUJO FUNCIONAL END-TO-END

El sistema **WebPrediccionDT2** implementa un ecosistema médico distribuido en tres capas principales: interfaz de usuario Blade/JavaScript (Frontend), capa de negocio y control Laravel 11 (Backend), y motor de inferencia analítica dual (Microservicio Python Random Forest y Google Gemini 1.5 Flash para asistencia diagnóstica). A continuación se audita el flujo funcional ciclo por ciclo:

| Fase / Módulo                        | Tecnología & Ruta                                             | Descripción del Flujo                                                                                                                                                                                                                    | Estado Funcional                       |
|--------------------------------------|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|
| 1. Acceso & Sesión                   | /login<br>/register                                           | Autenticación por correo y contraseña con Hash bcrypt. El registro público asigna automáticamente el rol Paciente (idrol=4) y genera un registro temporal en la tabla paciente.                                                          | <b>OPERATIVO</b><br>(Falta Rate Limit) |
| 2. Citas & Triage                    | /citas<br>/trajes/create                                      | Recepción médica y enfermería. Se registran signos vitales (glucosa, presión, IMC, edad, etc.). La cita pasa a estado 'Triado' para ser atendida por el médico.                                                                          | <b>OPERATIVO</b>                       |
| 3. Inferencia ML (Random Forest)     | /predicciones/create<br>\$ ightarrow\$ appml-tesis.vercel.app | El médico consulta los datos precargados del triaje. Al pulsar 'Predecir Diabetes', Laravel efectúa una llamada HTTP POST al microservicio Flask con los 8 parámetros normalizados. Devuelve probabilidad soft y diagnóstico preliminar. | <b>OPERATIVO</b>                       |
| 4. Asistencia IA Generativa          | /predicciones/analizar-gemini                                 | Analiza antecedentes clínicos y adjuntos (PDF/DOCX/JPG). Si GEMINI_API_KEY no está presente en .env, arroja excepción controlada.                                                                                                        | <b>REQUIERE KEY</b><br>(En .env)       |
| 5. Validación Médica (Instrumento 3) | /predicciones{id}/validar                                     | El especialista valida interactivamente el diagnóstico clínico real (Sí=Diabético / No=Sano) mediante AJAX con token CSRF. Sincronizado con los 80 pacientes evaluados en Casa Grande.                                                   | <b>OPERATIVO</b><br>(Accuracy 70.0%)   |
| 6. Auditoría & Matriz Confusión      | /doctores/costos<br>/confusion/export                         | Cálculo del costo operacional por doctor (COPG) y cálculo global de Matriz de Confusión en tiempo real: TP=19, TN=37, FP=6, FN=18 a umbral óptimo 0.55 con exportación a Excel.                                                          | <b>OPERATIVO</b>                       |

## 2. AUDITORÍA DE SEGURIDAD Y BRECHAS VULNERABLES (OWASP TOP 10)

Se realizó un escaneo estático y dinámico de arquitectura, código fuente en controladores, rutas, middlewares y configuraciones. A continuación se detallan las vulnerabilidades críticas, altas y moderadas identificadas:

| ID   | Vulnerabilidad Identificada                                         | Clasificación OWASP                | Severidad | Impacto Clínico                                            |
|------|---------------------------------------------------------------------|------------------------------------|-----------|------------------------------------------------------------|
| V-01 | Falta de Middleware RBAC en Rutas Administrativas                   | A01:2021 Broken Access Control     | CRÍTICA   | Escalación de privilegios de Paciente a Administrador      |
| V-02 | Insecure Direct Object Reference (IDOR) en Pacientes y Predicciones | A01:2021 Broken Access Control     | CRÍTICA   | Fuga de expedientes clínicos de terceros por ID            |
| V-03 | Exfiltración de Reportes Clínicos vía Endpoint SendEmail            | A01:2021 Broken Access Control     | ALTA      | Envío no autorizado de historial médico a correos externos |
| V-04 | Persistencia de Archivos Locales en Vercel Serverless (Read-Only)   | A05:2021 Security Misconfiguration | ALTA      | Fallo 500 al subir avatares y pérdida de adjuntos en /tmp  |
| V-05 | Entorno con Depuración Activa (APP_DEBUG=true)                      | A05:2021 Security Misconfiguration | ALTA      | Fuga de credenciales de Supabase y rutas de servidor       |
| V-06 | Microservicio ML sin Autenticación y CORS Abierto (*)               | A07:2021 Auth Failures             | MEDIA     | Consumo no autorizado de cómputo y riesgo de denegación    |

|      |                                                                 |                                 |       |                                                               |
|------|-----------------------------------------------------------------|---------------------------------|-------|---------------------------------------------------------------|
| V-07 | Ausencia de Rate Limiting en Autenticación (/login)             | A07:2021 Auth Failures          | MEDIA | Ataques de fuerza bruta y credential stuffing                 |
| V-08 | Renderizado HTML de IA sin Sanitización ({!! \$analysis_ia !!}) | A03:2021 Injection (Stored XSS) | BAJA  | Posible inyección XSS si un adjunto malicioso engaña a Gemini |

DETALLE DE BRECHAS Y PRUEBAS DE CONCEPTO TÉCNICAS

**V-01: Broken Access Control — Falta de Middleware de Roles (SEVERIDAD: CRÍTICA)**

**Evidencia en Código:** En routes/web.php, el grupo completo de rutas administrativas (/users, /roles, /users/create, /users/{id}/edit) se encuentra protegido únicamente por el middleware auth:

```
Route::middleware(['auth'])->group(function () { Route::get('/users', [IndexController::class, 'users']); ... });
```

**Mecánica de Vulnerabilidad:** Cuando un paciente se registra en /register, obtiene el rol 4. Si el paciente navega a /users, el sistema lista a todos los médicos y administradores. El paciente puede editar a cualquier administrador o cambiar su propio idrol a 1 (Admin), logrando escalada de privilegios total.

**Remediación:** Crear un Middleware RoleMiddleware y aplicarlo como ['auth', 'role:admin'] sobre las rutas críticas.

**V-02: Insecure Direct Object References (IDOR) en Expedientes (SEVERIDAD: CRÍTICA)**

**Evidencia en Código:** En PrediccionController@show(\$idprediccion), @edit y @downloadAttachment(\$id, \$index):

```
$prediccion = Prediccion::findOrFail($id); return view('predicciones.show', compact('prediccion'));
```

**Mecánica de Vulnerabilidad:** No se valida si el Auth::user()->id pertenece al doctor tratante ni al paciente objeto del reporte. Un usuario autenticado puede solicitar secuencialmente /predicciones/1 hasta /predicciones/80 o descargar archivos clínicos adjuntos de cualquier persona.

**Remediación:** Implementar Políticas de Autorización (Gate::authorize('view', \$prediccion)) verificando que el usuario sea el doctor o paciente asociado.

**V-03: Fuga de Datos Clínicos por Exfiltración en SendEmail (SEVERIDAD: ALTA)**

**Evidencia en Código:** En PrediccionController@sendEmail(Request \$request, \$idprediccion):

```
$request->validate(['email' => 'required|email']); Mail::send(..., function($m) use ($request, $pdf) { $m->to($request->email)->attachData(...); });
```

**Mecánica de Vulnerabilidad:** El endpoint permite ingresar cualquier correo de destino sin validar destinatario legítimo. Cualquier usuario puede exfiltrar el informe médico confidencial en PDF de cualquier paciente hacia casillas de correo externas no reguladas.

**Remediación:** Enviar reportes únicamente a la dirección de correo registrada en el perfil del paciente o doctor autenticado.

**V-04: Conflicto de Almacenamiento Local en Vercel Serverless (SEVERIDAD: ALTA)**

**Evidencia en Código:** En PacienteController@update:

```
$path = $file->move(public_path('images/pacientes'), $nombreImagen);
```

**Mecánica del Problema:** Las funciones Serverless en Vercel montan un sistema de archivos de SOLO LECTURA. Intentar escribir en public\_path() arrojará RuntimeException: Read-only file system (Error 500). Además, los adjuntos en /tmp desaparecen tras reciclar la instancia lambda.

**Remediación:** Configurar el controlador de almacenamiento para utilizar **Supabase Storage Bucket** vía API S3 o REST.

|                                                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>V-05: Entorno con APP_DEBUG=true</b>                                                                                                                                                                                                                          | <b>V-06: Microservicio ML con CORS Libre (*)</b>                                                                                                                                                                                                   |
| <p><b>Riesgo:</b> En caso de excepción no capturada en Vercel, Laravel despliega la pantalla Ignition exponiendo contraseñas de base de datos, Session Pooler de Supabase y tokens.</p> <p><b>Solución:</b> Configurar APP_DEBUG=false y APP_ENV=production.</p> | <p><b>Riesgo:</b> app.py tiene CORS(origins='*') sin API Key. Cualquier sitio externo puede ejecutar inferencias abusivas agotando recursos del servidor.</p> <p><b>Solución:</b> Restringir CORS al dominio web y validar cabecera X-API-Key.</p> |

3. AUDITORÍA DE BASE DE DATOS SUPABASE & RLS

Se auditó la configuración de seguridad a nivel de motor PostgreSQL en la instancia Supabase (Session Pooler):

| Componente de Base de Datos | Configuración Detectada                                                                 | Diagnóstico de Seguridad                                                                                                  | Recomendación Técnica                                                                               |
|-----------------------------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Row Level Security (RLS)    | Habilitado en las 16 tablas públicas (relrowsecurity=True), con 0 políticas declaradas. | <b>PROTEGIDO POR DEFECTO</b><br>La API REST anónima devuelve [] vacío ante intentos de lectura de pacientes/predicciones. | Mantener RLS activo. Si se usa cliente JS en frontend, declarar políticas estrictas por auth.uid(). |

|                          |                                                                                          |                                                                                                                                            |                                                                                          |
|--------------------------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Conexión Laravel Backend | Usuario postgres.wtmvupwxo<br>vhqgvvozwer conectando a<br>través del Pooler puerto 5432. | <b>BYPASS RLS LEGÍTIMO</b><br>El superusuario postgres opera con<br>privilegios de sistema, permitiendo<br>que Laravel controle la lógica. | Preservar credenciales en .env<br>fuera del repositorio git y forzar<br>sslmode=require. |
| Integridad Instrumento 3 | 80 registros actualizados con<br>probabilidades exactas y<br>validación médica pretest.  | <b>100% CONSISTENTE</b><br>TP=19, TN=37, FP=6, FN=18,<br>Exactitud=70.0% idéntica a Tabla 11<br>de la tesis.                               | Datos validados en Supabase y en<br>la vista /doctores/costos con<br>umbral 0.55.        |

## 4. PLAN DE REMEDIACIÓN INMEDIATA & CÓDIGO SUGERIDO

Para subsanar las brechas de seguridad identificadas antes de la sustentación o pase a producción, se recomienda ejecutar el siguiente plan de remediación:

- Paso 1: Implementar Middleware de Control de Roles (RBAC)**

Crear el archivo `app/Http/Middleware/CheckRole.php` que valide: `if (!in_array(Auth::user()->idrol, $roles)) abort(403, 'Acceso Denegado');` y aplicarlo en `routes/web.php` aislando las rutas `/users`, `/roles` para administradores (`idrol=1`) y `/doctores` para personal médico.
- Paso 2: Mitigar IDOR mediante Laravel Policies**

Generar `PrediccionPolicy` y `PacientePolicy` vinculadas a los modelos. En `PrediccionController@show`, invocar `$this->authorize('view', $prediccion);` para impedir que pacientes descarguen o visualicen registros ajenos.
- Paso 3: Blindar el Envío de Correos**

En `PrediccionController@sendEmail`, remover el campo libre `$request->email` y sustituirlo por el correo del paciente registrado: `$destinatario = $prediccion->cita->paciente->usuario->email` ?? `$prediccion->cita->paciente->email;`
- Paso 4: Endurecimiento de Variables de Entorno en Vercel**

En el dashboard de Vercel (Settings  $\rightarrow$  Environment Variables), configurar obligatoriamente: `APP_ENV=production`, `APP_DEBUG=false`, `SESSION_SECURE_COOKIE=true`, y registrar la `GEMINI_API_KEY` para el análisis IA.
- Paso 5: Proteger el Microservicio ML (Flask)**

En `appml_tesis/app.py`, restringir el CORS: `CORS(app, resources={r'/predict': {'origins': ['https://tudominio.vercel.app']}})` y validar un token de cabecera: `if request.headers.get('X-API-KEY') != os.getenv('ML_SECRET_KEY'): return jsonify({'error': 'Unauthorized'}), 401.`

## 5. MATRIZ DE MADUREZ FUNCIONAL Y CONCLUSIONES

| Criterio Evaluado              | Calificación (1-5) | Nivel de Madurez | Comentarios del Auditor                                                                                                           |
|--------------------------------|--------------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Flujo Clínico & Diagnóstico ML | ■■■■■ (5/5)        | Excelente        | Pipeline Random Forest integrado, datos del Instrumento 3 con 70.0% de exactitud y visualización de KPIs operacionales impecable. |
| Rendimiento & Conexión BD      | ■■■■■ (4/5)        | Robusto          | El Session Pooler de Supabase evita agotamiento de sockets en entornos serverless y resuelve latencias.                           |
| Arquitectura de Despliegue     | ■■■■■ (4/5)        | Adecuada         | Serverless con Vercel PHP 0.7.3 y redirección a /tmp. Requiere migrar subida de avatares a almacenamiento de objetos en la nube.  |
| Seguridad & Control de Acceso  | ■■■ (2/5)          | Vulnerable       | Falta control de roles a nivel de rutas y protección IDOR en expedientes. Debe subsanarse con las directivas del Capítulo 4.      |

**Conclusión Final del Auditor:** El sistema **WebPrediccionDT2** posee una base funcional y matemática de excelencia para los objetivos de la tesis, con algoritmos de aprendizaje automático coherentes y datos reales plenamente consistentes con los instrumentos validados. Sin embargo, dado el carácter sensible de la información clínica, la aplicación de los parches de control de acceso (RBAC) y la desactivación de depuración en producción son imperativos para garantizar una postura de seguridad robusta y alineada con los estándares de ingeniería de software médico.

Área de Auditoría de Sistemas y Seguridad

Ingeniería de Software & Ciberseguridad

Tesista / Líder de Proyecto

Sistema WebPrediccionDT2